



**MEDIACREST**  
TRAINING COLLEGE

# Machine Learning II: Principal Component Analysis (PCA)

Week 8, Day 1: Unsupervised Learning & Dimensionality Reduction

Victor Wandera Lumumba, MSc.

Statistician | Data Analyst | ML Expert

Mediacrest Training College

5th Oct 2026

# Abstract

This presentation explores Principal Component Analysis (PCA), a fundamental unsupervised learning technique for dimensionality reduction.

- **Theoretical Foundation:** Understanding how PCA transforms correlated variables into uncorrelated principal components while maximizing variance retention.
- **Data Preprocessing:** The critical importance of standardization before applying PCA to prevent scale bias.
- **Component Selection:** Using Scree plots and cumulative explained variance to determine the optimal number of components.
- **Interpretation & Application:** Analyzing component loadings to understand underlying data structures and evaluating the impact of PCA on Logistic Regression performance.

# Session Objectives

- Define Principal Component Analysis (PCA) and its role in unsupervised learning.
- Explain the mathematical intuition behind eigenvalues, eigenvectors, and variance maximization.
- Demonstrate the step-by-step workflow: Standardization  $\rightarrow$  Covariance Matrix  $\rightarrow$  Eigendecomposition.
- Interpret PCA results, including explained variance ratios and component loadings.
- Compare machine learning model performance (Logistic Regression) using raw features versus PCA-transformed features.

## Part 1: Theoretical Foundations

*"Reducing complexity while preserving information."*

# What is Dimensionality Reduction?

- **Definition:** The process of reducing the number of random variables under consideration by obtaining a set of principal variables.
- **Why is it needed?**
  - **Curse of Dimensionality:** High-dimensional data can lead to overfitting and increased computational cost.
  - **Multicollinearity:** Highly correlated features can destabilize linear models.
  - **Visualization:** Humans cannot visualize data beyond 3 dimensions; reduction allows 2D/3D plotting.
- **Two Main Approaches:**
  - **Feature Selection:** Selecting a subset of relevant features.
  - **Feature Extraction:** Creating new composite features (e.g., PCA).

# Introduction to Principal Component Analysis (PCA)

## Definition

PCA is a multivariate statistical technique that transforms a large set of correlated variables into a smaller set of uncorrelated variables called **Principal Components (PCs)**.

- **Goal:** Reduce dimensionality while retaining as much information (variance) as possible.
- **Key Property:** The new variables (PCs) are orthogonal (uncorrelated) to each other.
- **Applications:** Data compression, noise filtering, exploratory data analysis, and preprocessing for supervised learning.

# Objectives of PCA

- **Reduce Variables:** Simplify the dataset by decreasing the number of features.
- **Remove Multicollinearity:** Eliminate redundancy among predictors by creating independent components.
- **Retain Variation:** Ensure the new components capture the maximum possible variance from the original data.
- **Simplify Interpretation:** Make complex high-dimensional data easier to visualize and understand.
- **Improve Efficiency:** Enhance computational speed and stability in downstream predictive modeling.

# How PCA Works: The Concept

- PCA creates new variables (Principal Components) that are linear combinations of the original variables.
- **First Principal Component (PC1):** Captures the largest possible variance in the data.
- **Second Principal Component (PC2):** Captures the second-largest variance and is orthogonal (perpendicular) to PC1.
- **Subsequent Components:** Explain progressively smaller amounts of variance.
- **Mathematical Basis:** Relies on the decomposition of the covariance or correlation matrix using eigenvalues and eigenvectors.

# The Steps of PCA Algorithm

- ➊ **Standardize the Data:** Scale features to have mean 0 and variance 1 (critical for PCA).
- ➋ **Compute Covariance/Correlation Matrix:** Identify relationships between variables.
- ➌ **Calculate Eigenvalues and Eigenvectors:** Determine the directions of maximum variance.
- ➍ **Rank Principal Components:** Sort eigenvectors by their corresponding eigenvalues (highest to lowest).
- ➎ **Select Important Components:** Choose the top  $k$  components based on explained variance.
- ➏ **Transform Data:** Project the original data onto the new component space.

# Advantages of PCA

- **Reduces Complexity:** Simplifies datasets with many features.
- **Eliminates Redundancy:** Removes correlated information, preventing multicollinearity issues.
- **Facilitates Visualization:** Allows high-dimensional data to be plotted in 2D or 3D.
- **Enhances Model Performance:** Can improve accuracy and reduce training time for machine learning algorithms by removing noise.
- **Noise Reduction:** Discards components with low variance, which often represent noise.

# Limitations of PCA

## Critical Limitations

- **Interpretability:** Principal components are linear combinations of original features, making them harder to interpret physically.
- **Scale Sensitivity:** PCA is highly sensitive to the scaling of variables; standardization is mandatory.
- **Linearity Assumption:** PCA assumes linear relationships between variables. It may fail to capture complex non-linear structures.
- **Information Loss:** Some variance is always lost when reducing dimensions, though we aim to minimize this.

## Part 2: The CVD Risk Dataset

*"Exploring clinical data with high multicollinearity."*

# Loading Required Libraries

## Python Implementation

```
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5 from sklearn.preprocessing import StandardScaler
6 from sklearn.decomposition import PCA
7 from sklearn.model_selection import train_test_split, cross_val_score
8 from sklearn.linear_model import LogisticRegression
9 from sklearn.metrics import accuracy_score, roc_auc_score, classification_report
10 import warnings
11 warnings.filterwarnings('ignore')
12 plt.style.use('seaborn-v0_8-whitegrid')
13
```

## Code Explanation

- pandas/numpy: Data manipulation.
- sklearn.preprocessing.StandardScaler: Essential for normalizing data before PCA.

# Reading the Data

## Python Implementation

```
1 df = pd.read_csv("CVD_risk.csv")  
2 print(df.head())  
3
```

## Dataset Overview

The dataset contains clinical markers for Cardiovascular Disease (CVD) risk:

- **Features:** Systolic BP, Diastolic BP, Total Cholesterol, HDL, Fasting Glucose, CRP (Inflammation), Age, LDL.
- **Target:** CVD\_Risk (Binary: 0 = Low Risk, 1 = High Risk).
- **Observation:** Many clinical markers (e.g., BP, Cholesterol) are naturally correlated, making this an ideal candidate for PCA.

# Exploratory Data Analysis & Correlation

## Python Implementation

```
1 features = df.columns[:-1]
2 corr_matrix = df[features].corr()
3 plt.figure(figsize=(10, 7))
4 sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', vmin=-1, vmax=1)
5 plt.title('Correlation Matrix: High Multicollinearity Expected')
6 plt.show()
7
```

## Graph Explanation

- The heatmap visualizes pairwise correlations between all clinical features.
- **Observation:** High correlations ( $|r| > 0.7$ ) are observed among Blood Pressure, Cholesterol, and Glucose markers.
- **Implication:** This multicollinearity suggests redundancy. PCA is ideal here to reduce these correlated variables into independent components, stabilizing downstream models.

## Part 3: Data Preprocessing

*"Preparing data for variance-based algorithms."*

# Train/Test Split Strategy

## Python Implementation

```
1 X = df[features].values
2 y = df['CVD_Risk'].values
3
4 # Split BEFORE scaling to prevent data leakage
5 X_train, X_test, y_train, y_test = train_test_split(
6     X, y, test_size=0.25, random_state=42, stratify=y)
7
```

## Code Explanation

- **Leakage Prevention:** We split the data *before* scaling. If we scaled the whole dataset first, information from the test set would "leak" into the training statistics (mean/std).
- `test_size=0.25`: 25% of data is held out for final evaluation.
- `stratify=y`: Ensures the proportion of High/Low risk patients is preserved in both train and test sets, crucial for balanced evaluation.

# Feature Standardization

## Python Implementation

```
1 scaler = StandardScaler()  
2 # Fit ONLY on training data  
3 X_train_scaled = scaler.fit_transform(X_train)  
4 # Transform test data using training parameters  
5 X_test_scaled = scaler.transform(X_test)  
6  
7 print("    Data split & standardized (mean=0, variance=1).")  
8
```

## Why Standardize?

- PCA is **variance-sensitive**. Features with larger scales (e.g., Age in years vs. Cholesterol in mg/dL) would dominate the principal components if not scaled.
- StandardScaler transforms each feature to have a mean of 0 and standard deviation of 1.
- We fit the scaler on X\_train only, then apply those same parameters to X\_test to maintain consistency.

## Part 4: PCA Fitting & Component Selection

*"Identifying the hidden structures in data."*

# Fitting PCA on All Components

## Python Implementation

```
1 # Initialize PCA with all components to evaluate variance
2 pca_full = PCA(random_state=42)
3 X_train_pca_full = pca_full.fit_transform(X_train_scaled)
4
5 print(f"Shape of transformed data: {X_train_pca_full.shape}")
6
```

## Code Explanation

- We initialize `PCA()` without specifying `n_components`, so it computes all possible components (equal to the number of features).
- `fit_transform`:
  - 1 Calculates the covariance structure and extracts eigenvectors/eigenvalues.
  - 2 Projects the standardized training data onto these new axes.
- This step allows us to inspect how much variance each component explains before deciding how many to keep.

# Explained Variance Ratio

## Python Implementation

```
1 explained_var = pca_full.explained_variance_ratio_  
2 cumulative_var = np.cumsum(explained_var)  
3  
4 print(f"Explained Variance per Component: {np.round(explained_var, 3)}")  
5 print(f"Cumulative Variance: {np.round(cumulative_var, 3)}")  
6
```

## Output Interpretation

- **Explained Variance:** Shows the proportion of total dataset variance captured by each individual PC.
- **Cumulative Variance:** The running total of variance explained.
- **Example Output:**
  - PC1 explains 66.2% of variance.
  - PC2 adds 16.7% (Total: 82.9%).
  - PC3 adds 12.4% (Total: 95.2%).

# Determining Optimal Components

## Python Implementation

```
1 threshold = 0.90 # Target 90% variance retention
2 n_components_opt = np.argmax(cumulative_var >= threshold) + 1
3
4 print(f"           Optimal Components for {threshold*100}% Variance: {n_components_opt}")
5
```

## Logic Explanation

- We set a threshold (e.g., 90%) to retain most information while reducing dimensions.
- `np.argmax(cumulative_var >= threshold)` finds the index of the first component where cumulative variance exceeds 90%.
- **Result:** For this dataset, **3 components** are sufficient to explain 95.2% of the total variance.
- This reduces the feature space from 8 variables to just 3, significantly simplifying the model.

# Visualizing Variance (Scree Plot)

## Python Implementation

```
1 fig, axes = plt.subplots(1, 2, figsize=(14, 5))
2
3 # Left: Scree Plot
4 axes[0].plot(range(1, len(explained_var)+1), explained_var, 'bo-', label='Individual'
5             )
6 axes[0].plot(range(1, len(cumulative_var)+1), cumulative_var, 'ro-', label='
7             Cumulative')
8 axes[0].axhline(y=0.90, color='g', linestyle='--', label='90% Threshold')
9 axes[0].axvline(x=n_components_opt, color='purple', linestyle='--', label=f'Optimal k
10             ={n_components_opt}')
11 axes[0].set_title('Scree Plot: Explained Variance')
12 axes[0].legend()
```

## Graph Explanation

- The **Scree Plot** displays individual variance (blue) and cumulative variance (red).
- The green dashed line marks the 90% threshold

# 2D PCA Projection Visualization

## Python Implementation

```
1 # Right: 2D Projection
2 pca_2d = PCA(n_components=2)
3 X_train_2d = pca_2d.fit_transform(X_train_scaled)
4
5 scatter = axes[1].scatter(X_train_2d[:, 0], X_train_2d[:, 1], c=y_train, cmap='
        coolwarm', alpha=0.6)
6 axes[1].set_title('2D PCA Projection of Training Data')
7 axes[1].set_xlabel('PC1')
8 axes[1].set_ylabel('PC2')
9 plt.colorbar(scatter, ax=axes[1])
10 plt.tight_layout()
11 plt.show()
12
```

## Graph Explanation

- This plot projects the high-dimensional data onto the first two principal components.
- Points are colored by the target variable (CVD Risk)

# Final PCA Transformation

## Python Implementation

```
1 # Fit final PCA with optimal components
2 pca_opt = PCA(n_components=n_components_opt, random_state=42)
3 X_train_pca = pca_opt.fit_transform(X_train_scaled)
4 X_test_pca = pca_opt.transform(X_test_scaled)
5
6 print(f"Original Features: {X_train_scaled.shape[1]}")
7 print(f"PCA Components: {X_train_pca.shape[1]}")
8
```

## Code Explanation

- We re-initialize PCA with `n_components=3`.
- `fit_transform` on training data learns the 3 specific axes.
- `transform` on test data projects it onto these same 3 axes.
- The dataset is now reduced from 8 dimensions to 3, ready for modeling.

## Part 5: Component Interpretation

*"What do these new components actually mean?"*

# Understanding Component Loadings

- **Loadings:** The weights assigned to each original feature in the linear combination that forms a Principal Component.
- **Interpretation:**
  - High absolute loading ( $> 0.5$  or  $< -0.5$ ) indicates the feature strongly influences that component.
  - Positive/Negative signs indicate the direction of the relationship.
- By examining loadings, we can assign semantic meaning to the abstract PCs (e.g., "PC1 represents Metabolic Risk").

# Calculating and Displaying Loadings

## Python Implementation

```
1 loadings = pd.DataFrame(  
2     pca_opt.components_.T,  
3     columns=[f'PC{i+1}' for i in range(n_components_opt)],  
4     index=features  
5 )  
6 print(loadings.round(3))  
7
```

## Code Explanation

- `pca_opt.components_` contains the eigenvectors.
- We transpose (`.T`) the matrix so that rows are features and columns are PCs.
- This creates a readable table showing how much each clinical marker contributes to PC1, PC2, and PC3.

# PCA Loadings Table

| Feature           | PC1    | PC2    | PC3    |
|-------------------|--------|--------|--------|
| Systolic_BP       | 0.403  | -0.257 | 0.040  |
| Diastolic_BP      | 0.403  | -0.257 | 0.055  |
| Total_Cholesterol | 0.421  | 0.109  | 0.014  |
| HDL_Cholesterol   | -0.396 | 0.275  | -0.027 |
| Fasting_Glucose   | 0.387  | 0.316  | -0.003 |
| CRP_Inflammation  | 0.128  | 0.812  | -0.070 |
| Age               | -0.044 | 0.089  | 0.995  |
| LDL_Cholesterol   | 0.416  | 0.116  | -0.002 |

**Table:** Component Loadings: Feature weights in each Principal Component

## Key Insight

High absolute values indicate strong influence. PC1 is driven by cholesterol/BP, PC2 by Inflammation, and PC3 by Age.

# Interpreting PC1: Cardiometabolic Risk

- **Dominant Features:**

- Total Cholesterol (0.421)
- LDL Cholesterol (0.416)
- Systolic BP (0.403)
- Diastolic BP (0.403)
- Fasting Glucose (0.387)
- HDL Cholesterol (-0.396)

- **Interpretation:** PC1 represents a general **Cardiometabolic Risk Axis**.

- High values in PC1 correspond to high blood pressure, high bad cholesterol, high glucose, and low good cholesterol (HDL).
- This single component captures the combined effect of major cardiovascular risk factors.

# Interpreting PC2: Inflammation

- **Dominant Feature:**
  - CRP Inflammation (0.812)
- **Secondary Features:**
  - Fasting Glucose (0.316)
  - HDL Cholesterol (0.275)
- **Interpretation:** PC2 primarily captures **Systemic Inflammation**.
- CRP (C-Reactive Protein) is a marker of inflammation. Its high loading suggests this component isolates inflammatory variation from the general metabolic risk captured in PC1.
- There is a moderate link to metabolic dysregulation (Glucose/HDL).

# Interpreting PC3: Age

- **Dominant Feature:**
  - Age (0.995)
- **Other Features:** Negligible loadings ( $< 0.1$ ).
- **Interpretation:** PC3 is almost entirely an **Age Dimension**.
- This indicates that age varies independently of the biochemical markers (cholesterol, inflammation) in this dataset.
- It confirms that age provides unique information not captured by the metabolic or inflammatory components.

# Visualizing Loadings Heatmap

## Python Implementation

```
1 plt.figure(figsize=(8, 5))
2 sns.heatmap(loadings, annot=True, cmap='coolwarm', center=0, fmt='.2f')
3 plt.title('PCA Loadings: Original Features      Principal Components')
4 plt.tight_layout()
5 plt.show()
6
```

## Graph Explanation

- The heatmap visually maps the strength and direction of feature contributions.
- **Blue/Red Contrast:** Highlights positive vs. negative correlations.
- **Pattern Recognition:**
  - Column 1 (PC1) shows strong red/blue blocks for lipids and BP.
  - Column 2 (PC2) highlights CRP.
  - Column 3 (PC3) highlights Age.
- This visualization confirms the distinct clinical constructs identified by PCA

## Part 6: Model Evaluation

*"Does PCA improve prediction? Raw vs. PCA Features."*

# Baseline Model: Logistic Regression (Raw)

## Python Implementation

```
1 # Train on original scaled features
2 model_raw = LogisticRegression(max_iter=1000, random_state=42)
3 model_raw.fit(X_train_scaled, y_train)
4
5 # Predictions
6 y_pred_raw = model_raw.predict(X_test_scaled)
7 y_prob_raw = model_raw.predict_proba(X_test_scaled)[: , 1]
8
```

## Code Explanation

- We train a Logistic Regression model on the **original 8 scaled features**.
- This serves as our **baseline** to compare against the PCA-reduced model.
- `max_iter=1000` ensures convergence.
- We generate both class predictions (`predict`) and probabilities (`predict_proba`) for ROC-AUC calculation.

# Evaluating Baseline Model

## Python Implementation

```
1 cv_acc_raw = cross_val_score(model_raw, X_train_scaled, y_train, cv=5, scoring='
    accuracy').mean()
2 cv_auc_raw = cross_val_score(model_raw, X_train_scaled, y_train, cv=5, scoring='
    roc_auc').mean()
3
4 print(f"Test Accuracy: {accuracy_score(y_test, y_pred_raw):.4f}")
5 print(f"Test ROC-AUC : {roc_auc_score(y_test, y_prob_raw):.4f}")
6 print(f"CV Accuracy   : {cv_acc_raw:.4f}")
7 print(f"CV ROC-AUC    : {cv_auc_raw:.4f}")
8
```

## Code Explanation

- **Test Metrics:** Evaluate performance on unseen data.
- **Cross-Validation (CV):** Provides a more robust estimate of model stability by averaging performance over 5 folds of the training data.
- We track both **Accuracy** (correct classifications) and **ROC-AUC** (discriminatory power).

# Baseline Results: Raw Scaled Features

| Metric        | Score  |
|---------------|--------|
| Test Accuracy | 0.8400 |
| Test ROC-AUC  | 0.9252 |
| CV Accuracy   | 0.8600 |
| CV ROC-AUC    | 0.9452 |

**Table:** Performance of Logistic Regression on Raw Scaled Features

## Classification Report Summary

- **Low Risk:** Precision 0.85, Recall 0.83.
- **High Risk:** Precision 0.83, Recall 0.85.
- The model is balanced and performs well, but uses all 8 original features.

# PCA Model: Logistic Regression (PCA)

## Python Implementation

```
1 # Train on PCA-transformed features (3 components)
2 model_pca = LogisticRegression(max_iter=1000, random_state=42)
3 model_pca.fit(X_train_pca, y_train)
4
5 # Predictions
6 y_pred_pca = model_pca.predict(X_test_pca)
7 y_prob_pca = model_pca.predict_proba(X_test_pca)[: , 1]
```

## Code Explanation

- We train the **same** Logistic Regression algorithm, but this time on the **3 PCA components**.
- The input data is now much simpler (3 columns instead of 8).
- This tests if the reduced feature set retains enough information to maintain or improve predictive performance.

# Evaluating PCA Model

## Python Implementation

```
1 cv_acc_pca = cross_val_score(model_pca, X_train_pca, y_train, cv=5, scoring='accuracy')
2 cv_auc_pca = cross_val_score(model_pca, X_train_pca, y_train, cv=5, scoring='roc_auc')
3
4 print(f"Test Accuracy: {accuracy_score(y_test, y_pred_pca):.4f}")
5 print(f"Test ROC-AUC : {roc_auc_score(y_test, y_prob_pca):.4f}")
6 print(f"CV Accuracy   : {cv_acc_pca:.4f}")
7 print(f"CV ROC-AUC    : {cv_auc_pca:.4f}")
8
```

## Code Explanation

- We use the same metrics (Accuracy, ROC-AUC, CV) for fair comparison.
- Note that we use `X_train_pca` and `X_test_pca` (the transformed data).
- This allows us to directly compare the efficacy of the raw features vs. the engineered PCA features.

# Final Comparison and Conclusion

| Metric        | Raw Features (8) | PCA Features (3) |
|---------------|------------------|------------------|
| Test Accuracy | 0.8400           | <b>0.8533</b>    |
| Test ROC-AUC  | 0.9252           | 0.9200           |
| CV Accuracy   | 0.8600           | 0.8467           |
| CV ROC-AUC    | 0.9452           | 0.9302           |

Table: Performance Comparison: Raw vs. PCA-Transformed Features

## Conclusion

- **Improved Accuracy:** PCA model achieved higher test accuracy (85.3% vs 84.0%), suggesting better generalization by removing noise.
- **Efficiency:** Reduced features from 8 to 3, lowering computational cost.
- **Interpretability:** Identified 3 key drivers: Metabolic Risk, Inflammation, and Age.
- **Verdict:** PCA successfully simplified the dataset while maintaining robust predictive performance, making it a valuable preprocessing step for clinical risk modeling.